

问题 A. 区间深度

与具有所有不同元素的连续子数组中的前两个位置相对应的可能对的数量与数组的长度呈线性关系。更具体地，以下引理成立。

引理。

令 N 为正整数，令 P 为 $(1, 2, \dots, N)$ 的任意排列。对于每个 $1 \leq L \leq R \leq N$ ，定义整数 a 和 b ，使得 $P_L, P_{L+1}, \dots, P_R$ 中最小元素和第二小元素分别为 P_a 和 P_b 。那么，以这种方式可以出现的对 (a, b) 的总数是 $\Theta(N)$ 。

这是因为所有具有 $a \neq b$ 的对 (a, b) 上的半开区间集 $[\min(a, b), \max(a, b))$ 是层流的。

首先，枚举所有可能的对 (a, b) 。从 $(L, R) = (1, N)$ 开始，我们可以递归地缩小区间 (L, R) ，使得对应的对 (a, b) 发生变化，并递归地处理这些区间。这可以使用线段树或类似的数据结构在 $O(N \log N)$ 时间内完成。

接下来，当根据其定义递归计算 $f(L, R)$ 时，请注意，在递归深度至少为 1 时，可能的 (L, R) 对的数量为 $O(N)$ 。因此，记住已计算的 $f(L, R)$ 值以避免冗余计算是有效的。使用这种方法，我们为所有可能的对 (a, b) 预先计算 $f(\min(a, b) + 1, \max(a, b) - 1) + 1$ 。

然后，对于每个可能的对 (a, b) ，我们计算与其对应的间隔 (L, R) 。这可以通过计算包含闭区间 $[\min(a, b), \max(a, b)]$ 并且不包含除 P_a 之外任何小于 P_b 的元素的闭区间 $[L, R]$ 的数量来完成。这可以使用线段树上的二分搜索或类似技术来有效地计算。

最后，对于 $k = f(\min(a, b) + 1, \max(a, b) - 1) + 1$ ，我们将此类间隔的数量 (L, R) 添加到 k 的答案中。通过对所有可能的 (a, b) 执行此操作，我们获得了 $f(L, R)$ 在所有 (L, R) 上的分布，这正是我们需要计算的。

The 该算法的总体时间复杂度为 $O(N \log N)$ 。

问题 B. 增加掉期

此后，让 `left_rotate` 表示获取序列的第一个元素并将其附加到序列末尾的操作。相反，令 `right_rotate` 表示采用序列的最后一个元素并将其添加到开头的操作。

然后，单个操作包括将 `left_rotate` 应用于每个不相交的连续子数组。

此外，对于任何 t ，如果一个元素在第 t 次操作中属于同一连续子数组，则它也必须属于第 $(t + 1)$ 次操作中属于同一连续子数组。

设 $Q = P^{-1}$ ，并考虑以相反的顺序执行操作，旨在按升序对 Q 进行排序。然后，每个操作都包括将 `right_rotate` 应用于几个不相交的连续子数组中的每一个。

我们引入以下符号：

- 如果序列 a 是通过按顺序连接序列 a_1 和 a_2 获得的，我们写为 $a = a_1 + a_2$ 。
- 令 $f(a)$ 表示序列 a 的解值。
- 如果满足以下条件，则称序列 $a = a_1 + a_2 + \dots + a_\ell$ 可拆分为 $a_1, a_2, \dots, a_\ell$ ：

◦ a_1 是 $(1, \dots, |a_1|)$ 的排列，◦ a_2 是 $(|a_1| + 1, \dots, |a_1| + |a_2|)$ 的排列，◦ ...

◦ a_ℓ 是 $(\sum_{i=1}^{\ell-1} |a_i| + 1, \dots, \sum_{i=1}^{\ell} |a_i|)$ 的排列。

在这个定义下，下面的贪心策略是最优的（稍后给出证明）：

- 重复应用 `right_rotate` 直到序列第一次可分为至少两个部分。
- 分割序列，使得分割后的部分数量最大化，然后独立求解每个结果子序列。
 - 也就是说，如果 $a = a_1 + a_2 + \dots + a_\ell$ 可拆分为 $a_1, a_2, \dots, a_\ell$ ，则 $f(a) = \max\{f(a_1), f(a_2), \dots, f(a_\ell)\}$ 。

因此，可以通过重复应用 `right_rotate` 直到序列可分割，然后递归地解决每个子问题来解决该问题。

此外，可以使用下面描述的算法在相对于序列长度的线性时间内确定序列首先变得可分割之前所需的 `right_rotate` 操作的数量，并且也可以在线性时间内计算实际的分割。由于递归深度最多为 N ，并且在每个深度我们总共处理 N 个元素，因此总体时间复杂度为 $O(N^2)$ 。

查找序列第一个之前的 `right_rotate` 操作次数的算法
可拆分为两个或更多部分

t

令 $x = (x_1, x_2, \dots, x_N)$ 为 $(1, 2, \dots, N)$ 的排列。

我们按升序处理这些值。由于值集是 $\{1, 2, \dots, N\}$ ，因此可以在 $O(N)$ 时间内完成。准备一个二元数组 $A = (A_1, \dots, A_N)$ ，它被认为是循环数组，即 A_N 的右邻居是 A_1 。

- 最初， A 中的所有值都设置为 0。
- 对于 $i = 1, \dots, N - 1$ ，执行以下操作：
 - 查看所有位置 j ，以便 $Q_j = i$ （按升序处理值），并将 $A_j \leftarrow$ 设置为 1。
 - 如果 A 中值为 1 的所有位置形成单个连续间隔，则该序列可拆分，第一个段的长度为 i 。通过维护 0 和 1 相邻的索引，我们可以在 $O(1)$ 时间内确定序列是否可拆分以及必须应用 `right_rotate` 多少次。
- 在针对 $i = 1, \dots, N - 1$ 获取的所需 `right_rotate` 操作次数的值中，返回最小值。

证明贪婪策略是最优的

我们首先证明以下引理。

引理

对于任何序列 a 及其子序列 a' ，我们有 $f(a') \leq f(a)$ 。

- 执行 `right_rotate` 操作时，提取子序列会导致对该子序列执行 `right_rotate` 操作，或者不会发生任何变化。

- 可以通过将该间隔上的操作开始延迟一步来消除不执行任何操作的操作。
- 因此，从 a 的排序序列，我们可以构造一个不再使用任何操作的 a' 的排序序列。

根据这个引理，我们继续进行证明。

对于长度为 n 的序列 $a = a_1 + a_2$ ，只要证明如果 a 可拆分为 a_1 和 a_2 ，则 $f(a) = \max(f(a_1), f(a_2))$ 。（如果 a 不可拆分，则无论如何我们都必须将 `right_rotate` 应用于整个序列，因此考虑可拆分情况就足够了。）

$n = 1, 2$ 的情况很简单。假设该语句对于所有长度 $m \leq n - 1$ 都成立。

令 t 为长度为 n 的可分割序列，并令 t' 为在对 t 应用一次或多次 `right_rotate` 后第一次变得可分割的序列。我们根据分割 t 得到的前导连续段的长度和分割 t' 得到的前导连续段的长度之间的关系来区分情况。

情况1：长度相等

在这种情况下，从 t 和 t' 获得的前导段是同一组元素的排列。由于 t' 是通过重复的 `right_rotate` 操作获得的，因此只有在 $t = t'$ 时才会发生这种情况。因此，该声明是微不足道的。

情况2： t 的前导段长度小于 t' 的前导段长度

让

$$t = t_1 + t_2 + t_3 + t_4, \quad t' = t_4 + t_1 + t_2 + t_3,$$

并假设分割 t 得到的前导段是 t_1 ，而分割 t' 得到的前导段是 t_1

是

$t_4 + t_1 + t_2$ 。我们可以假设 $|t_1|, |t_3|, |t_4| > 0$ 和 $|t_2| \geq 0$ 。

然后：

- 直接拆分 t 会得到以下解

$$\max(f(t_1), f(t_2 + t_3 + t_4)).$$

- 将 `right_rotate` 应用于 t 直到它变成 t' ，然后分裂产生以下解决方案

$$|t_4| + \max(f(t_4 + t_1 + t_2), f(t_3)).$$

考虑 $f(t_2 + t_3 + t_4)$ 。在 $|t_4|$ 应用 `right_rotate` 之后，序列可以分为 $t_4 + t_2$ 和 t_3 。如果它可以用更少的操作进行拆分，这将与 t' 的定义相矛盾。自 $|t_2 + t_3 + t_4| < n$ 以来，我们有

$$f(t_2 + t_3 + t_4) = |t_4| + \max(f(t_4 + t_2), f(t_3)).$$

所以，

$$\begin{aligned} & \max(f(t_1), f(t_2 + t_3 + t_4)) \\ &= \max(f(t_1), |t_4| + \max(f(t_4 + t_2), f(t_3))) \\ &\leq |t_4| + \max(f(t_1), f(t_4 + t_2), f(t_3)) \\ &\leq |t_4| + \max(f(t_4 + t_1 + t_2), f(t_3)). \end{aligned}$$

因此，该主张成立。

情况3: t 的前导段长度大于 t' 的前导段长度

让

$$t = t_4 + t_1 + t_2 + t_3, \quad t' = t_1 + t_2 + t_3 + t_4,$$

并假设分割 t 得到的前导段是 $t_4 + t_1 + t_2$, 而分割得到的前导段是 $t_4 + t_1 + t_2$
 t' 是 t_1 。然后我们有:

g

- 直接拆分 t 会得到以下解

$$\max(f(t_4 + t_1 + t_2), f(t_3)).$$

- 将 `right_rotate` 应用于 t 直到它变成 t' , 然后分裂产生以下解决方案

$$|t_1| + |t_2| + |t_3| + \max(f(t_1), f(t_2 + t_3 + t_4)).$$

由于序列 $t_4 + t_1 + t_2$ 在 $|t_1| + |t_2|$ 操作之后可拆分为 t_1 和 $t_2 + t_4$, 因此我们有

$$f(t_4 + t_1 + t_2) \leq |t_1| + |t_2| + \max(f(t_1), f(t_2 + t_4)).$$

所以,

$$\begin{aligned} & \max(f(t_4 + t_1 + t_2), f(t_3)) \\ & \leq \max(|t_1| + |t_2| + \max(f(t_1), f(t_2 + t_4)), f(t_3)) \\ & \leq |t_1| + |t_2| + |t_3| + \max(f(t_1), f(t_2 + t_4), f(t_3)) \\ & \leq |t_1| + |t_2| + |t_3| + \max(f(t_1), f(t_2 + t_3 + t_4)). \end{aligned}$$

因此, 该主张得到证明。

因此, 如果序列 t 是可拆分的, 则最好立即拆分它。通过数学归纳, 这适用于任何长度的序列。

问题 C. 三个反转之和

让 $Z = N - X - Y$ 。那么 Z 表示 A 中3出现的次数。另外, 定义 $D_i = (A_i, B_i, C_i)$ 。

对于具有 $1 \leq i < j \leq N$ 的整数 i, j , 将对 (i, j) 的贡献定义为以下三个成立的不等式的数量: $A_i > A_j$ 、 $B_i > B_j$ 和 $C_i > C_j$ 。根据这个定义, 问题可以重新表述为计算序列 D 的数量, 使得所有对 (i, j) 的总贡献恰好是 K 。

现在, 将 $(1, 2, 3)$ 的排列划分为以下两组:

- 组 P : $(1, 2, 3), (2, 3, 1), (3, 1, 2)$
- 组 Q : $(1, 3, 2), (2, 1, 3), (3, 2, 1)$

那么, 以下事实成立:

- 对于任何具有 $1 \leq i < j \leq N$ 的 i, j , 如果 D_i 和 D_j 属于不同组的排列, 则 (i, j) 的贡献恰好为 1。

这是因为当 (A_i, B_i, C_i) 和 (A_j, B_j, C_j) 是来自不同组的排列时, 三个等式 $A_i = A_j$ 、 $B_i = B_j$ 和 $C_i = C_j$ 中的一个恰好成立。如果 $A_i = A_j$, 则不等式 $B_i > B_j$ 和 $C_i > C_j$ 之一成立, 因此 (i, j) 的贡献为 1。情况 $B_i = B_j$ 和 $C_i = C_j$ 可以以相同的方式论证。

利用这一观察结果，所有对 (i, j) 的贡献总和可以表示为

$$(\text{组 } P \text{ 内的总贡献}) + (\text{组 } Q \text{ 内的总贡献}) + (\text{组 } P) \times (\text{的排列数 } Q) \cdot \text{的排列数}$$

将 $dp[x][y][z][k]$ 定义为仅由组 P 中的排列组成的序列数，使得排列 $(1, 2, 3)$ 出现 x 次， $(2, 3, 1)$ 出现 y 次， $(3, 1, 2)$ 出现 z 次，总贡献为 k 。这个 DP 表可以在 $O(N^5)$ 时间内计算出来。对于组 Q ，可以用相同的方式计算类似的 DP。

假设在 D 中， $(1, 2, 3)$ 、 $(2, 3, 1)$ 和 $(3, 1, 2)$ 的数量分别为 x 、 y 和 z ，并且在 P 组内的总贡献为 k 。那么以下数量有助于答案：

$$\binom{N}{x+y+z} \times dp[x][y][z][k] \times dp[X-x][Y-y][Z-z][K-k-(x+y+z)(N-x-y-z)].$$

通过对满足 $0 \leq x \leq X$ 、 $0 \leq y \leq Y$ 、 $0 \leq z \leq Z$ 和 $0 \leq k \leq K$ 的所有 x, y, z, k 求和该值，可以在 $O(N^5)$ 时间内计算出最终答案。

问题 D. 网格路径树

$N = 1$ 、 $M = 1$ 或 $k = 1$ 的情况很容易处理，因此我们省略它们。从现在开始，我们假设 $N \neq 1$ 、 $M \neq 1$ 和 $k \neq 1$ 。

缓慢的解决方案

首先，我们描述了一种计算固定值 k 的时间复杂度 $O(N + M)$ 答案的方法。树和从 $N \times M$ 网格的左上角移动到右下角的路径之间存在一一对应关系，仅使用向右移动和向下移动。

现在，在树上施加附加条件 $(a_2, b_2) = (2, N + 1)$ 。在这种情况下，这样的树与正整数序列一一对应

$$A = (A_1, A_2, \dots, A_{|A|})$$

满足以下条件：

$$N - 1 = A_1 + A_3 + \dots,$$

$$M - 1 = A_2 + A_4 + \dots.$$

这对应于向下移动 A_1 次，然后向右移动 A_2 次，然后向下移动 A_3 次，依此类推的路径。

对于由序列 A 表示的树，对 (i, j) 的数量使得 $\text{dist}(i, j) = k$ (对于 $k \neq 2$) 可以写为

$$\sum_{t=1, \dots} A_t A_{t+k-2}.$$

当 k 为奇数时，所有序列 A 和 $|A| = L$ 上的上述值的总和为

$$\max(0, L + 2 - k) \binom{N-1}{\lfloor \frac{L+1}{2} \rfloor} \binom{M-1}{\lfloor \frac{L}{2} \rfloor}.$$

当 k 为偶数时，可表示为

$$\max\left(0, \left\lfloor \frac{L+1}{2} \right\rfloor + 1 - k\right) \binom{N}{\lfloor \frac{L+1}{2} \rfloor + 1} \binom{M-2}{\lfloor \frac{L}{2} \rfloor - 1} + \max\left(0, \left\lfloor \frac{L}{2} \right\rfloor + 1 - k\right) \binom{N-2}{\lfloor \frac{L+1}{2} \rfloor - 1} \binom{M}{\lfloor \frac{L}{2} \rfloor + 1}.$$

当 $k = 2$ 时, 同样的公式也成立。

因此, 一旦确定了 k 和 $|A|$, 就可以在 $O(1)$ 时间内计算出答案。因此, 对于固定的 k , 可以在 $O(N + M)$ 时间内得到总答案。

当 $(a_2, b_2) = (1, N + 2)$ 时, 同样可以通过交换 N 和 M 来解决问题。

满分解法

对于任意 $2 \leq k \leq N + M - 5$, 该值

$$ans_{k+4} - 2 \cdot ans_{k+2} + ans_k$$

可以在 $O(1)$ 时间内计算, 假设二项式系数已预先计算。

因此, 直接使用上述方法计算出 $ans_2, ans_3, ans_4, ans_5$ 后, 可以利用这个事实适当地导出剩余值。

这样, 任意 k 的所有值 ans_k 都可以在总时间复杂度 $O(N + M)$ 中计算出来, 这足够快地解决这个问题。

e

问题 E. 最大两次子序列

$O(|B|)$ -每个查询的时间解决方案

首先, 让我们确定如何计算 $Q =$ 情况下的答案 1. 设 $n := |B|$, 并假设给定 $B = (B_1, B_2, \dots, B_n)$ 。

假设存在一个至少出现两次的子序列, 并采用两个不同的索引序列 $(i_1, i_2, \dots, i_k)$ 和 $(j_1, j_2, \dots, j_k)$ 来实现它。我们必须有 $k \geq 1$, 并且存在一些 p ($1 \leq p \leq k$) 使得 $i_p \neq j_p$ 。不失一般性, 假设 $i_p < j_p$ 。在这种情况下, $(i_1, i_2, \dots, i_{p-1}, i_p, j_{p+1}, j_{p+2}, \dots, j_k)$ 和 $(i_1, i_2, \dots, i_{p-1}, j_p, j_{p+1}, j_{p+2}, \dots, j_k)$ 也是产生相同子序列的不同索引序列。因此, 当我们寻找给出至少出现两次的子序列的不同索引序列时, 仅考虑仅在一个位置不同的那些索引序列对就足够了。

现在, 让我们用 $(i_1, i_2, \dots, i_a, l, j_1, j_2, \dots, j_b)$ 和 $(i_1, i_2, \dots, i_a, r, j_1, j_2, \dots, j_b)$ 表示这种不同的索引序列。在这里, 我们有

$$1 \leq i_1 < i_2 < \dots < i_a < l < r < j_1 < j_2 < \dots < j_b \leq n.$$

由于 $l \neq r$ 和 $B_l = B_r$, 序列 B 必须包含重复元素。更具体地说, 我们可以这样论证: 取最小的 s ($1 \leq s \leq n$), 使得后缀 $(B_s, B_{s+1}, \dots, B_n)$ 不包含重复元素。那么我们必须有 $l < s$, 如果 $s = 1$ 则不存在好的序列。

接下来, 考虑选择满足条件的索引序列 $(i_1, i_2, \dots, i_a, l, r, j_1, j_2, \dots, j_b)$, 从前面决定其索引。如果第一个索引是 i , 那么从 $l < s$ 我们得到

$$B_i = \max_{1 \leq t < s} B_t.$$

由于我们可能会从前面贪婪地选择子序列, 因此我们必须有

$$i = \min(\arg\max_{1 \leq t < s} B_t).$$

此时, 我们有以下两种选择:

1. 修复 $a \geq 1$ 并设置 $i_1 \leftarrow i$ 。当 $i <$ 时这是可能的 $s - 1$ 。

2. 修复 $a = 0$ 并设置

$$l \leftarrow i, \quad r \leftarrow \min\{t > i \mid B_t = B_i\}.$$

当 $\{t > i \mid B_t = B_i\} \neq \emptyset$ 时这是可能的。

无论哪种情况，注意到 $\{t \geq s \mid B_t = B_{s-1}\} \neq \emptyset$ ，我们都可以适当地选择剩余的索引来构建满足条件的索引序列。当以上两种选择都可进行时，我们必须决定采取哪一种。由于子序列的第一个元素 $C_1 = B_i$ 已经固定，因此下一个要最大化的数量是 C_2 。对于每个选项， C_2 给出如下：

1. 如果我们选择选项 1：我们正在有效地解决 $B \leftarrow B(i, n]$ 上的相同问题，所以

$$C_2 = \max_{i < t < s} B_t.$$

2. 如果我们选择选项 2：我们将从大于 r 的索引中贪婪地选择最大子序列

n

r ，因此

$$C_2 = \max_{r < t \leq n} B_t.$$

如果 C_2 的两个候选值不同，我们应该选择给出较大值的选项。如果它们相同，我们实际上应该选择选项 1。原因是，如果选择选项 2，并且从 r 之后的索引中取出的最大子序列是 X ，那么选择选项 1，并设置

$$a' = 1, \quad i'_1 = i, \quad l' = \min(\arg\max_{i < t < s} B_t), \quad r' = \min(\arg\max_{r < t \leq n} B_t),$$

我们可以得到相同的子序列 X 。在这里，我们可以看到 $l' < r'$ 如下：首先，如果是 $s \leq r'$ ，这从 $l' < s$ 可以清楚地看出。如果 $r' < s$ ，那么根据 r 和 r' 的定义，我们有 $i < r < r' < s$ ，并将 $[1, s]$ 上的 B_i 的最大值与 $B_i = B_r$ 相结合，我们得到

$$\max_{i < t < s} B_t = B_r.$$

将其与 l' 的定义进行比较，我们得到 $l' \leq r$ ，从而得到 $l' \leq r < r'$ ，这给出了所需的不等式。

现在我们已经得到了从前面选择索引序列的最优策略。在实现中，我们预先计算

- $s, , , ,$
- 托利姆 $i = \text{分钟}(\arg\max_{i < t < s} B_t)$,
- 结束 $i = \text{分钟}(\arg\max_{i < t \leq n} B_t)$,
- $\text{nxt}_i = \text{分钟}\{t > i \mid B_t = B_i\}$,

每当出现两个选项时，我们都会比较下一个值（ C_2 的候选值）并决定采用哪个选项。如果我们选择选项 1，我们继续执行相同的过程；如果我们选择选项 2，我们可以贪婪地从大于 r 的索引中获取最大子序列。所有这些都可以在 $O(|B|)$ 时间内完成。

以下是伪代码的实现。省略了输入、预处理和获取滚动哈希。

```
# B[1], ..., B[m] have been processed
m = 0
ans = []
while true :
    i = tolim[m]
    ans.append(B[i])
    if i == s-1 :
        m = nxt[i]
        break
    if nxt[i] != infty && B[tolim[i]] < B[toend[nxt[i]]] :
        m = nxt[i]
        break
    m = i
while m < n :
    i = toend[m]
    ans.append(B[i])
    m = i
```

加快速度

现在考虑将上述 $O(|B|)$ 解决方案应用于从查询 (L_i, R_i) 定义的 $B = (A_{L_i}, A_{L_i+1}, \dots, A_{R_i})$ 。我们看到解决方案中出现的 $s, \text{tolim}, \text{toend}$ 严重依赖于 R_i ，而几乎不依赖于 L_i 。因此，我们将提前读取所有查询并按 R_i 的升序处理它们。

假设已为 $B = (A_1, A_2, \dots, A_R)$ 计算了 $s, \text{tolim}, \text{toend}, \text{nxt}$ 。假设 $L < s$ 。如果我们重写上面的伪代码，使其适用于查询 (L, R) ，它就会变成：

```
# A[L], ..., A[m] have been processed
m = L-1
ans = []
while true :
    i = tolim[m]
    ans.append(A[i])
    if i == s-1 :
        m = nxt[i]
        break
    if nxt[i] != infty && A[tolim[i]] < A[toend[nxt[i]]] :
        m = nxt[i]
        break
    m = i
while m < R :
    i = toend[m]
    ans.append(A[i])
    m = i
```

假设我们可以在 $O(\log N)$ 时间内获取该伪代码中发生中断时的 i 的值；用 $i_{L,R}$ 表示该值。那么字典序上最大的好序列具有以下结构：

- 在 tolim 中出现的索引 i 中，按升序列出满足 $L \leq i < i_{L,R}$ 的索引，并将该序列称为 $(i_1, i_2, \dots, i_a)$ 。
- 在 toend 中出现的索引 j 中，按升序列出满足 $\text{nxt}_{i_{L,R}} < j \leq R$ 的索引，并将该序列称为 $(j_1, j_2, \dots, j_b)$ 。

- 字典顺序上最大的好序列正是 $(A_{i_1}, A_{i_2}, \dots, A_{i_a})$, $(A_{i_{L,R}})$ 和 $(A_{j_1}, A_{j_2}, \dots, A_{j_b})$ 按此顺序的串联。

因此, 如果我们准备存储单元素序列 (A_i) 的滚动哈希的线段树

对于分别出现在 `tolim` 或 `toend` 中的每个索引 i , 我们可以得到 $O(\log N)$ 中每个 L 的答案时间。

接下来, 请注意伪代码中的 `if` 条件不依赖于 L , 因此 $i_{L,R}$ 可以表征如下。固定 R , 并让 $L < s$ 任意。在 `tolim` 中出现的所有索引中, 只有那些满足伪代码中至少一个 `if` 条件的索引才能成为 $i_{L,R}$ 的候选索引。令 I_R 为此类索引的集合。那么 $i_{L,R}$ 是 I_R 中至少是 L 的最小元素。通过使用平衡二叉搜索树 (例如 `std::set`) 管理 I_R , 我们可以在 $O(\log N)$ 时间内获得 $i_{L,R}$ 。

剩下的任务是当 R 增加1时将 s , `tolim`, 更新为 `end`, `nxt`, 并对滚动哈希的线段树和 I_R 进行相应的更新。

首先, 随着 R 的增加, s 单调非减少。 `tolim` 和 `toend` 都随着 s 和 R 的增加而变化, 但其中出现的索引集可以使用堆栈来维护。当 R 增加1时, `nxt` 最多只有一项发生变化。至此, 忽略对 I_R 的更新, 所有所需的更新总共可以在 $O(N \log N)$ 时间内执行。

令 S 为 `tolim` 中出现的索引集合; 然后 $I_R \subseteq S$ 。我们将按如下方式更新 I_R 。通过在空集上适当地定义 `min` 和 `max` 并向 A 添加哨兵, 我们可以重写用于中断的 `if`, 以便将其统一为单个条件 $A[\text{tolim}[i]] < A[\text{toend}[\text{nxt}[i]]]$ 。为了便于阅读, 我们还将 A_* 写为 $A[*]$ 。

- 从 `tolim` 堆栈中弹出的任何索引 i 将不再是 I_R 的成员。

- 对于 $i \in S$, 定义

$$d_i = A[\text{toend}_{\text{nxt}_i}] - A[\text{tolim}_i].$$

那么 i 成为 I_R 的元素的条件是 $d_i > 0$ 。

- 当 `nxt` 更新时, 最多有一个索引 $i \in S \setminus I_R$ (即 $\text{nxt}_i = R$) 的索引的 d_i 增加, 并新成为 I_R 的元素。

- 当 `tolim` 更新时, 最多有一个索引 $i \in S$ (栈顶) 的 d_i 减少, 并且不再是 I_R 的元素。

- 当 `toend` 更新时, 对于那些 nxt_i 落入某个区间的 $i \in S \setminus I_R$, d_i 会增加, 并且最多只有一个成为 I_R 的元素。尤其:

– 如果对于该区间内的索引 i , nxt_i 不是 `toend` 的堆栈顶部, 则 $A[\text{nxt}_i] < A[\text{toend}_{\text{nxt}_i}]$ 成立。此外, 从 $i \in S$ 开始, 我们有 $A[i] \geq A[\text{tolim}_i]$ 。因此,

$$d_i = A[\text{toend}_{\text{nxt}_i}] - A[\text{tolim}_i] > A[\text{nxt}_i] - A[i] = 0,$$

因此, 这样的 i 成为 I_R 的元素。

由于 I_R 的元素不再属于 I_R 的次数总共可以受到 $O(N)$ 的限制, 因此即使我们在最后一项中天真地扫描 $S \setminus I_R$ 的相关部分, 整体复杂性也不会恶化。例如, 我们可以将 $S \setminus I_R$ 存储在平衡二叉搜索树中, 例如以 nxt_i 为键的 `std::set`, 并且对于给定范围, 只需遍历该范围内的所有元素。

通过仔细实施这些更新并充分注意它们的应用顺序, 我们可以解决这个问题。该解的时间复杂度为 $O((N + Q) \log N)$ 。

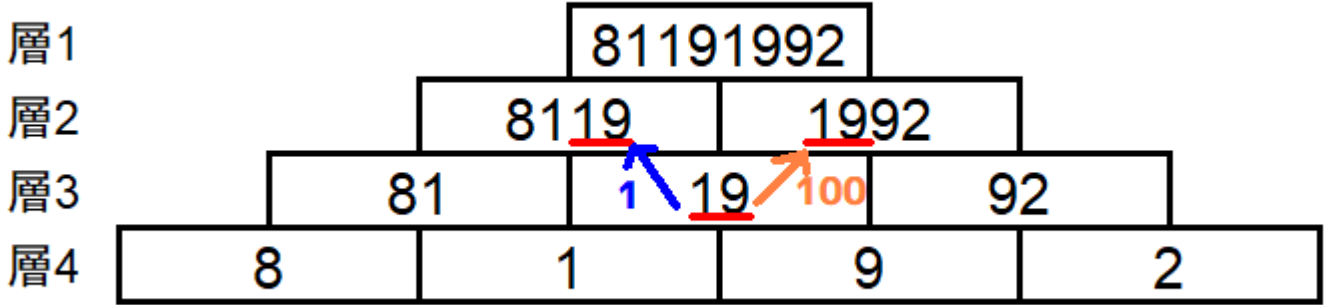
问题 F. 十进制金字塔

让我们考虑计算 t

b 层 N (中每个块 $C_{N,i}$ 的贡献

奥托姆层) 至 $C_{1,1}$ 。

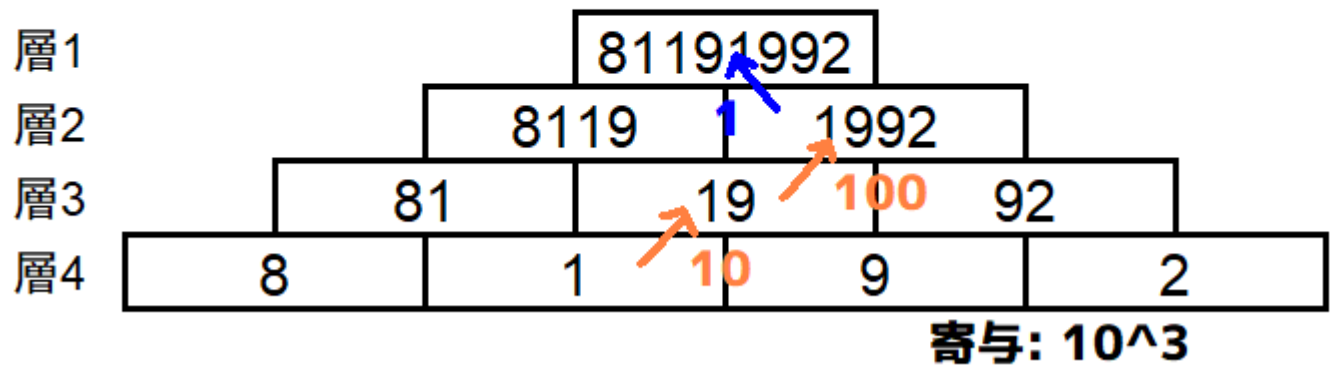
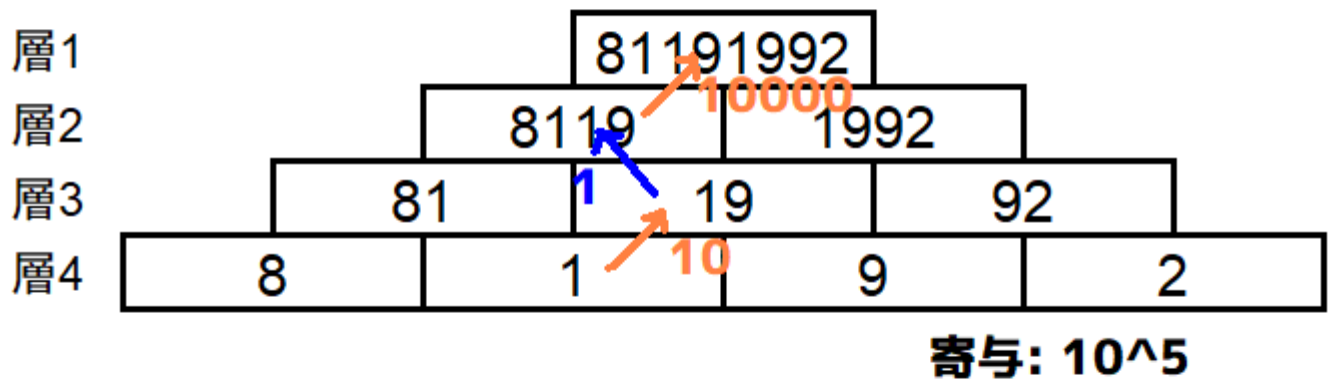
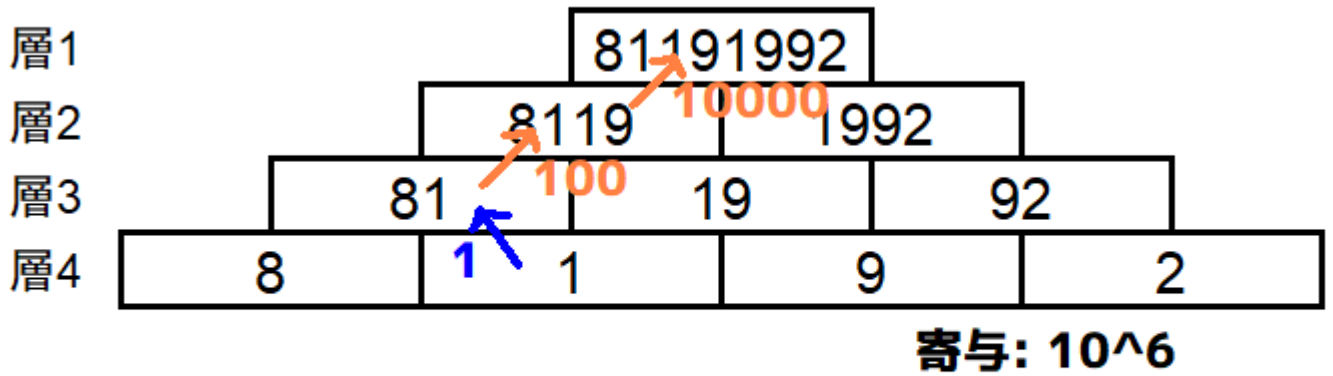
一般来说，块 $C_{i,j}$ 影响层 $i-1$ 中的两个块： $C_{i-1,j-1}$ 和 $C_{i-1,j}$ 。在此过程中，它对 $C_{i-1,j-1}$ 的贡献系数为1，对 $C_{i-1,j}$ 的贡献系数为 $10^{2^{N-i}}$ 。



当 $C_{i,j}$ 贡献给第 $i-1$ 层时，我们可以认为它有两个选择：贡献给 $C_{i-1,j-1}$ 或贡献给 $C_{i-1,j}$ 。我们称前一种选择为“移至左上方”，后一种选择为“移至右上方”。

通过反复做出“移动到左上”或“移动到右上”形式的贡献，我们最终将达到 $C_{1,1}$ 。可以看出，对于块 $C_{N,i}$ 来说，它“向左上”移动的次数正好是 $i-1$ ，而它“向右上”移动的次数正好是 $N-i$ 。

相反，任何由 $i-1$ 移动到左上角和 $N-i$ 移动到右上角组成的移动方式都表示从 $C_{N,i}$ 到 $C_{1,1}$ 的有效贡献。



现在，让我们用多项式来表达这一点。假设每次“移动到右上角”时，我们都会乘以 x 。那么，从 $C_{N,i}$ 到 $C_{1,1}$ 的总贡献可以写为

$$C_{N,i}[x^{N-i}] \prod_{t=1}^{N-1} (1 + 10^{2^{t-1}} x).$$

要计算所有 i 的值，只需展开多项式 $\prod_{t=1}^{N-1} (1 + 10^{2^{t-1}} x)$ 即可。这可以通过使用多项式合并技术（分而治之）在 $O(N \log^2 N)$ 时间内实现。

问题 G. 不要归零

事实上，长度为 X 的序列 $-(N - X - 1), -(N - X - 2), \dots, -A, A + 1, \dots, N - 1, N$ 始终是一个好的序列。

如果子序列中正整数的个数大于或等于负整数的个数，则子序列的和为正。如果正整数的数量小于负整数的数量，则总和为负数。

因此，当符号为 $-$ 时，我们可以按顺序附加 $-(N - X - 1), -(N - X - 2), \dots$ ，当符号为 $+$ 时，我们可以按顺序附加 $N, N - 1, \dots$ 。通过这样做，我们总能构建一个好的序列。

问题 H. Akari 计数

令 $h_1, h_2, h_3, w_1, w_2, w_3$ 为定义为 fo 的值 允许。

- $h_1 = A - 1$
- $h_2 = B - A + 1$
- $h_3 = H - B$
- $w_1 = C - 1$
- $w_2 = D - C + 1$
- $w_3 = W - D$

我们将网格划分为如下图所示的八个区域 A, B, C, D, E, F, G, H ，并让 a, b, c, d, e, f, g, h 分别表示放置在每个区域中的灯光数量。（请注意，从此时起， A, B, C, D 指的是区域名称。）

	w_1	w_2	w_3
h_1	E	A	F
h_2	B		D
h_3	G	C	H

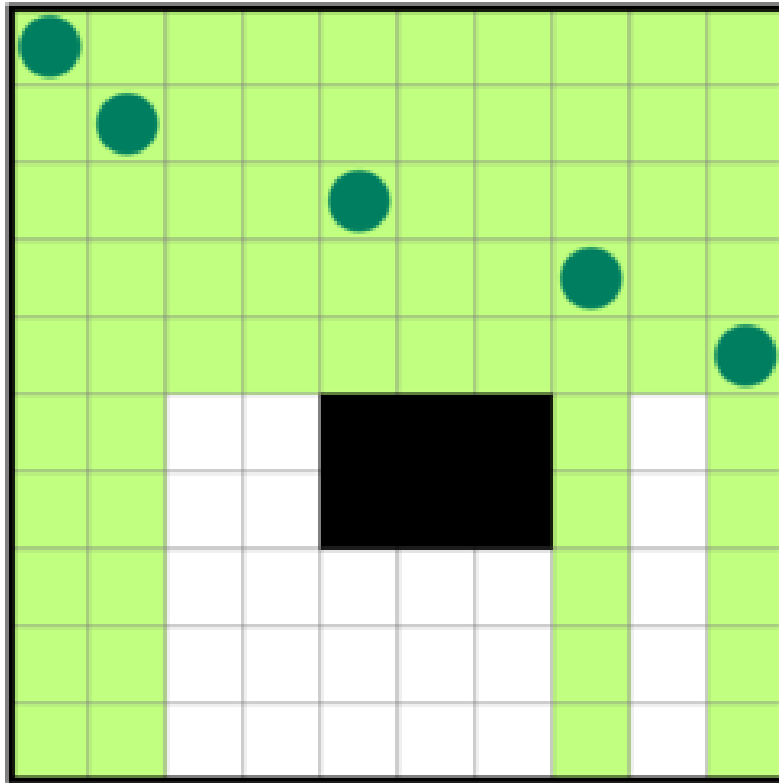
照亮所有白细胞的条件

区域 A 仅由放置在区域 A, E , 和 F 中的灯照亮。因此，为了照亮区域 A 中的所有单元，必须满足以下条件之一：

- $a + e + f = h_1$
- $a + e + f \neq h_1$ 和 $a = w_2$

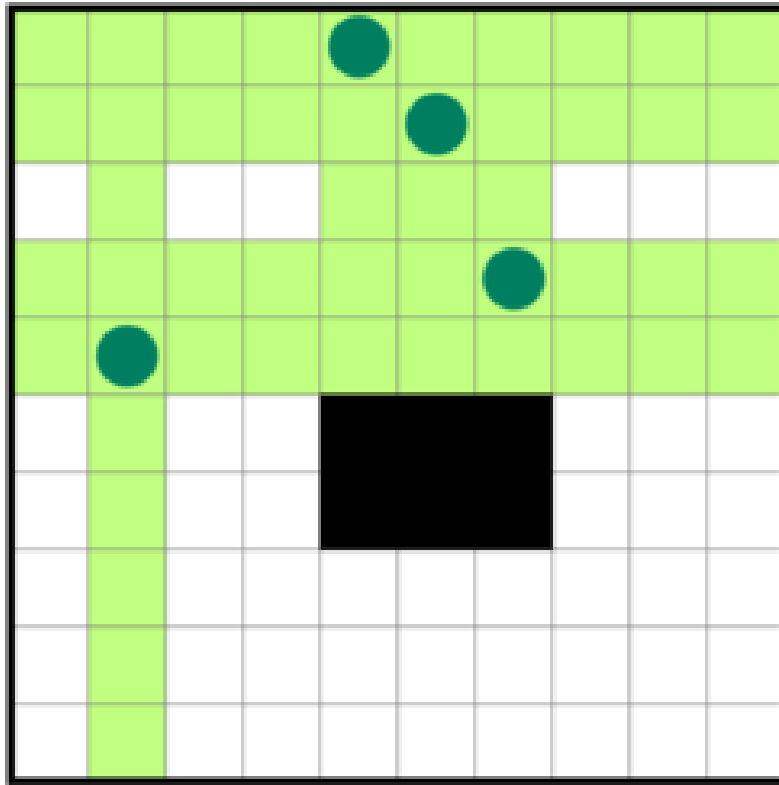
案例 $a + e + f = h_1$

区域 A , E , 和 F 中的所有单元都被照亮。



案例 $a = w_2$ (和 $a + e + f \neq h_1$)

区域 A 中的所有单元都被照亮，但区域 E 和 F 中的一些单元保持未照亮。这些剩余的单元必须由放置
在其他区域的灯垂直照亮。



S同样，考虑到区域 B 中所有白细胞的照明，必须满足以下条件之一：

- $b + e + g = w_1$
- $b + e + g \neq w_1$ 和 $b = h_2$

接下来，考虑照亮区域 E 中的所有白细胞。

假设最上面的 h_1 行中有一行没有放置灯光，最左边的 w_1 列中也有一列没有放置灯光。那么该行和列交叉处的白细胞就没有被任何光照亮，违反了条件。考虑到这一点，为了照亮区域 E 中的所有白细胞，必须至少满足以下条件之一：

- $a + e + f = h_1$
- $b + e + g = w_1$

将相同的推理应用于区域 F, G , 和 H ，我们看到为了照亮所有白细胞区域 E, F, G , 和 H ，必须满足以下条件之一：

在

- $a + e + f = h_1$ and $d + g + h = h_3$
- $b + e + g = w_1$ and $d + f + h = w_3$

基于这些观察，我们着手解决问题。

计数

首先，我们部分修复满足照明区域 A, B, C , 和 D 条件的灯光位置。然后，我们确定剩余的位置，使得照亮区域 E, F, G , 和 H 的条件也得到满足。

到照明 inate 区域 A 中的所有单元，以下 mus 之一 保持：

- $a + e + f = h_1$
- $a + e + f \neq h_1$ 和 $a = w_2$

令 $X_0[i]$ 为在区域 A 中放置灯光的方式数量，使得 $a + e + f = h_1$ 和 $e + f = i$ 。该值由 $\binom{w_2}{h_1-i} \frac{h_1!}{i!}$ 给出。

接下来，考虑 $a + e + f \neq h_1$ 、 $a = w_2$ 和 $e + f = i$ 的情况。我们计算由区域 A 中的灯光位置以及区域 E 和 F 中至少包含一个灯光的一组行组成的对的数量。这个数字是 $\binom{h_1-w_2}{i} \cdot \frac{h_1!}{(h_1-w_2)!}$ ，我们用 $X_1[i]$ 表示它。

对区域 C, G , 和 H 应用相同的计算，我们定义 $Y_0[i]$ 和 $Y_1[i]$ 。

然后我们定义 $P_0[i]$ 和 $P_1[i]$ 如下：

- $P_0[i] = \sum_{j+k=i} X_0[j] \cdot Y_0[k]$
- $P_1[i] = \sum_{j+k=i} (X_0[j] \cdot Y_1[k] + X_1[j] \cdot Y_0[k] + X_1[j] \cdot Y_1[k])$

$P_0[i]$ 和 $P_1[i]$ 的含义如下：

- $P_0[i], P_1[i]$: 区域 A 和 D 中带有灯光的白细胞以及区域 E, F, G , 和 H 中含有灯光的行集的固定方式数，行集的大小为 i
- $P_0[i]$: $a + e + f = h_1$ 和 $d + g + h = h_3$ 均成立
- $P_1[i]$: $a + e + f \neq h_1$ 或 $d + g + h \neq h_3$ 至少之一成立

同样，我们对列进行相同的分析，并定义 $Q_0[i]$ 和 $Q_1[i]$ 如下：

- $Q_0[i], Q_1[i]$: 区域 B 和 C 中带有灯光的白细胞以及区域 E, F, G , 和 H 中含有灯光的列集的固定方式数，列集的大小为 i
- $Q_0[i]$: $b + e + g = w_1$ 和 $c + f + h = w_3$ 都成立
- $Q_1[i]$: $b + e + g \neq w_1$ 或 $c + f + h \neq w_3$ 至少之一成立

现在考虑将 i 灯光精确放置在区域 E, F, G , 和 H 中。一旦要使用的行和列的集合被固定，放置灯光的方法的数量就是 $i!$ 。

因此，最终答案可以表示为：

$$\sum_i (P_0[i] \cdot Q_0[i] + P_0[i] \cdot Q_1[i] + P_1[i] \cdot Q_0[i]) \cdot i!$$

计算的瓶颈在于评估 P_0, P_1, Q_0 , 和 Q_1 。通过使用卷积，这可以
时间复杂度为 $O((H + W) \log(H + W))$ 。

n

问题 I. 子网格连通分量

分析

直接计算连接组件数量的简单方法将花费 $O(N^2Q)$ 时间，这太慢了。相反，我们观察到该图具有网格结构并寻找有用的结构属性。

首先，请注意该网格形成平面图。因此，它有几个由边包围的区域（我们也将无界的外部区域视为一个区域）。

如果我们删除包围某个区域的一条边，则连通分量的数量不会改变，而区域的数量会减少 1。重复这个过程，如果原来的区域数量为 r ，我们可以删除 $r - 1$ 条边，使区域数量等于 1。

接下来，如果我们在两个不同的连通分量之间添加一条边（这是可能的，因为图形是网格形状的），区域的数量不会改变，而连通分量的数量会减少 1。因此，如果原始连通分量的数量为 a ，我们可以添加 $a - 1$ 条边来使图连通。

生成的图是连通的并且没有环，即它是一棵树。因此，如果顶点数为 v ，则边数必须为 $v - 1$ 。

令 e 为原始边数，我们得到

$$e - (r - 1) + (a - 1) = v - 1,$$

可以重新安排

$$a = v - e + r - 1.$$

这个公式也可以理解为欧拉多面体定理的一个实例。

从这个观察中，我们看到每个查询都可以通过计算三个数量来回答：
顶点数、边数和区域数。

r

如问题中所述，顶点数为 $((D_i - U_i + 2)/2) \times ((R_i - L_i + 2)/2)$ 。边的数量可以使用边的前缀和来计算，从而允许每个查询使用 $O(N^2)$ 预处理时间和 $O(1)$ 时间。困难的部分是计算区域的数量。

计算区域数量

为了解决这个问题，我们使用对偶图的概念。在对偶图中，原图的每个区域对应一个顶点。考虑偶坐标 (x, y) 表示的点。如果两个这样的点相邻并且它们之间没有边缘，则认为它们属于同一区域。而且，如果一个点与外部相连，则认为该点属于外部区域。

对于查询给出的子网格，完全包含在子网格中的任何闭环也仍然是该子网格内的闭合区域。

仅当原始网格具有循环时区域的行为才会发生变化，但子网格仅包含该循环的一部分，导致区域连接到外部。这种情况仅发生在子网格的外部边界上，并且最多有 $4N$ 个这样的点。

回答“子网格包含多少个周期？”的问题有效地，我们分配
各地区的代表点。

e

该方法的工作原理如下：

- 如果子网格完全包含环，则它必须包含该区域的代表点。
- 如果子网格根本不包含环，则它不包含任何代表点。

- 如果子网格仅包含循环的一部分，则可以包含也可以不包含代表点。然而，通过详尽地检查沿边界与外部连接的部分，我们可以删除与外部连接的区域对应的所有代表点。如上所述，我们只需要检查 $4N$ 点。

这样我们就可以准确的统计出循环次数了。通过预处理 $O(N^2)$ 时间内所有区域的代表点的前缀和，我们可以在 $O(N)$ 时间内回答每个查询。

总的来说，这产生了一个具有 $O(N^2)$ 预处理时间和空间以及每个查询 $O(N)$ 时间的算法。

问题 J. 划分多边形

多边形剖分与树之间的对应关系

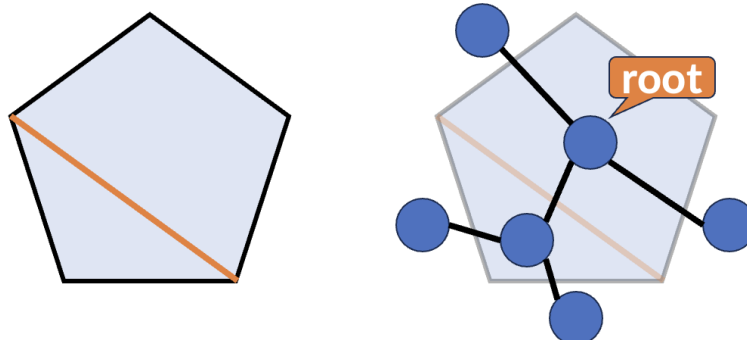
剖析多边形的方法与满足以下所有条件的有根树一一对应。顶点未标记，但每个顶点的子节点都是有序的。

- 正好有 $N - 1$ 个叶子。
- 正好有 $k + 1$ 个非叶顶点，包括根。
- 对于子节点数量为 c 的非叶顶点，值 $c + 1$ 包含在 S 中。

建立解剖和树之间对应关系的一种可能方法如下。这里，令线段 i 表示连接顶点 i 和顶点 $i + 1$ 的边，线段 N 表示连接顶点 N 和顶点 1 的边。

- 对于解剖中生成的每个多边形，关联一个非叶顶点。
- 对于每个段 i ($1 \leq i \leq N - 1$)，关联一个叶子。
- 当且仅当对应的多边形共享相同的对角线时，通过边连接两个顶点。
- 当且仅当多边形包含线段 i ($1 \leq i \leq N - 1$) 时，连接相应的顶点。
- 选择包含线段 N 的多边形对应的顶点作为根。
- 对子级进行排序，以便叶子按照与段 $1, 2, \dots, N - 1$ 对应的顺序从左到右出现。

下图显示了 $N = 5$ 和 $k = 1$ 的示例，其中通过绘制连接顶点 2 和 4 的对角线以及相应的树来剖切多边形。



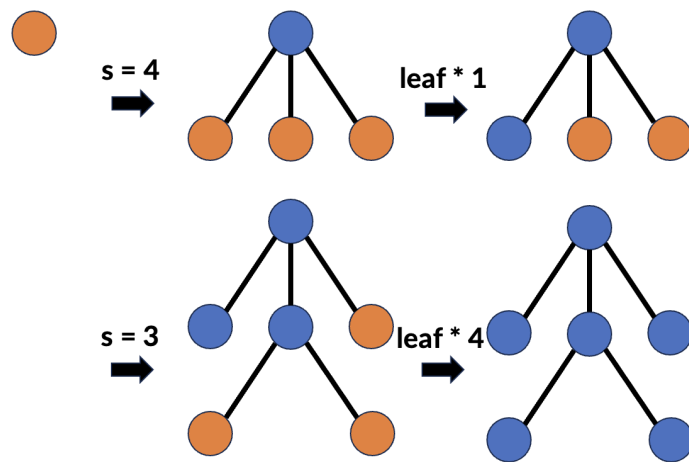
树和序列之间的对应关系

上述树可以通过从由单个根组成的树开始并精确地执行以下操作 $N + k$ 次来构建。

- 选择并执行操作 1 或 2 之一。操作 1 必须总共执行 $N - 1$ 次。
 - 动作1：选择最左边的叶子并将其固定为叶子。
 - 操作 2：选择 $s \in S$ ，对于尚未固定为叶子的最左边的顶点，向其添加 $s - 1$ 个子节点。

例如，对应于前面所示的五边形剖分的树可以通过以下操作顺序。

e



我们将构造树的操作序列与整数序列 A 关联起来，如下所示

。

- 从 A 为空开始，从左到右扫描操作，修改 A 如下。
 - 执行操作 1 时，将 -1 附加到 A 的末尾。
 - 执行操作 2 时，将 $s - 2$ 附加到 A 的末尾。

对于图中所示的示例，这对应于 $A = (2, -1, 1, -1, -1)$ 。

当且仅当操作序列有效时，最终序列 A 满足以下所有条件。

- $|A| = N + k$
- A 中的每个元素 a 满足 $a = -1$ 或 $(a + 2) \in S$
- A 中所有元素的总和为 -1
- 对于任何 $1 \leq i < N + k$ ，我们有 $A_1 + A_2 + \dots + A_i \geq 0$

因此，满足上述所有条件的有效序列 A 的个数就是答案。

计算序列

如果我们忽略最后一个条件，则有效序列的数量可以使用 $f(x) = \sum_{s \in S} x^{s-2}$ 写为 $\binom{N+k}{k+1} [x^{N-2}] f(x)^{k+1}$ 。将该值记为 C_k 。

即使添加了最后一个条件，通过应用伯特兰选票定理，我们可以看到答案变为 $\frac{C_k}{N+k}$ 。

更具体地说，这可以通过在以下两组之间建立一一对应关系来表明：

- 对 (A, x) ，其中 A 满足包括最后一个在内的所有条件，而 x 是具有 $0 \leq x < N+k$ 的整数
- 满足除最后一个条件之外的所有条件的序列 B

定义对应关系，将 B 作为将 A 向左 b 循环移位得到的序列

x 职位。例如，对 $A = (2, -1, 1, -1, -1)$ 和 $x = 3$ 对应于 $B = (-1, -1, 2, -1, 1)$ 。因此，对于每个 $0 \leq k \leq N-3$ ，计算 $\frac{C_k}{N+k}$ 就足够了。

计算的瓶颈在于评估所有 $0 \leq k \leq N-3$ 的 C_k 。这个问题被称为功率投影，可以在时间复杂度 $O(N \log^2 N)$ 内求解。

因此，这个问题的答案也可以在相同的时间复杂度内计算出来。

问题 L. 彩色四边形

0. 定义

- 对于两个 2D 向量 $\vec{u}, \vec{v}$ ，将叉积定义为

$$\vec{u} \times \vec{v} := u_x v_y - v_x u_y.$$

- 对于 2D 向量 $\vec{v}$ ，将其逆时针旋转 90° 定义为

$$\text{rot}(\vec{v}) := (-v_y, v_x).$$

- 对于 2D 向量 $\vec{v}$ ，将 $\text{top4}(\vec{v})$ 定义为以下四个点的集合：对于每种颜色 c ，选择与 $\vec{v}$ 内积最大化的颜色 c 的点。在所有选定的点中，取与 $\vec{v}$ 内积最大的四个点。

1. 最优四边形的性质

令最优四边形为 $ABCD$ 。考虑仅已知两个点 A 和 C 的情况。那么点 B 和 D 可以确定如下：

1. 忽略与 A 和 C 颜色相同的点。2. 让 $\vec{v} := \text{rot}(\overrightarrow{AC})$ 。选择与 $\vec{v}$ 内积最小的点作为 B ，与最大内积的点作为 D 。
3. 如果 B 和 D 具有相同的颜色，则将其中一个替换为剩余颜色中的最小/最大点（以产生较大面积的为准）。

4. 如果 e 四边形是简单四边形（非自相交），其面积是

$$\frac{1}{2}(\overrightarrow{AC} \times \overrightarrow{BD}).$$

如果是自相交，可以将 A, B, C, D 重新排序以形成面积更大的简单四边形，因此使用 $\frac{1}{2}(\overrightarrow{AC} \times \overrightarrow{BD})$ 是可以接受的。

从第 2 步到第 3 步我们有：

- 对于每种颜色 c ，令 P_c 为颜色 c 的点集。不在 P_c 凸包上的任何点都不会在答案中被选择。
- 对于每个选定的点 X ，都存在某个方向 $\vec{v}$ 使得 $X \in \text{top4}(\vec{v})$ 。

2. $O(N^3)$ 解决方案

1. 枚举所有对 (A, C) 。
2. 通过使用 $\text{rot}(\overrightarrow{AC})$ 获取最小/最大内积，在 $O(N)$ 时间内计算 B 和 D)。

3. 快速查询：颜色范围内的最大内积

使用类似线段树的结构，我们可以在 $O((\log N)^2)$ 时间内找到 $[L, R]$ 中与方向 $\vec{v}$ 内积最大化的颜色点。

线段树的每个节点存储颜色在该区间内的点的凸包。给定 $O(\log N)$ 节点和每个节点的 $O(\log N)$ 时间来查询最大内积，我们每次查询都会获得 $O((\log N)^2)$ 。

如果方向 $\vec{v}$ 已排序，我们可以预先计算最大化顶点发生变化的断点，并将成本降低到 $O((N + Q) \log N)$ 。

4. $O(N^2 \log N)$ 算法

1. 在 $O(N^2 \log N)$ 时间内枚举所有 $\overrightarrow{AC}$ 并按极角排序。
2. 对于每个 $\overrightarrow{AC}$ ，使用分摊 $O(\log N)$ 时间内的数据结构找到与 A 和 C 颜色不同的 B, D 。
3. 如果 B 和 D 具有相同的颜色，请重新选择一个以使面积最大化。

5. 恰好四种颜色的情况

假设四边形中的四种颜色依次为 1, 2, 3, 4 A, B, C, D 。令 $\overrightarrow{AC}$ 的方向为 $\vec{v}$ 。然后：

- 对于颜色 2，最大化 $-\text{rot}(\vec{v})$ 的点是 B 。
- 对于颜色 4，最大化 $\text{rot}(\vec{v})$ 的点是 D 。

注意：以下说法不正确：

- For color 1, the point maximizing $-\vec{v}$ is A
- For color 3, the point maximizing $\vec{v}$ is C

Hence the problem can be solved in $O(N \log N)$ time:

1. Sort points by x -coordinate.
2. Compute convex hulls of each color and discard interior points.
3. Fix A of color 1 and enumerate C of color 3.
4. Rotate $\vec{v}$ over $[0, 360^\circ)$ and enumerate candidates for $\overrightarrow{BD}$. This corresponds to taking the Minkowski sum of -1 times the hull of color 2 and the hull of color 4, which takes $O(N)$ time.
5. Similarly enumerate candidates for $\overrightarrow{AC}$.
6. Using two-pointer scanning over sorted directions, find the $\overrightarrow{BD}$ that maximizes $|\overrightarrow{AC} \times \overrightarrow{BD}|$.

6. Randomized Approach

If there are more than four colors, randomly map the colors to $\{1, 2, 3, 4\}$ and run the above algorithm. With probability $1/32$, the largest-area quadrilateral is preserved. Repeating yields high success probability.

Because all repeatable steps take $O(N)$, the complexity is $O(N \log N + RN)$ for R repetitions.

Since the problem contains multiple test cases, a purely randomized solution may not achieve AC, but combining with the $O(N^3)$ method for small N may work.

7. Number of Distinct $\text{top4}(\vec{v})$

We study how many different $\text{top4}(\vec{v})$ arise as $\vec{v}$ rotates through 360° .

When colors are all distinct, the number is $O(N)$ due to results on order- k Voronoi diagrams:

$$O(k(N - k)) \quad \text{with } k = 4 \text{ or } k = N - 4.$$

Reference: Lee, Der-Tsai. "On k -nearest neighbor Voronoi diagrams in the plane." IEEE Transactions on Computers 100.6 (1982): 478–487.

For general colored setting, the number is still $O(N)$. The essential idea:

If we can show that the possible sets of colors appearing in $\text{top4}(\vec{v})$ are at most $7N$, then the number of top4 types is at most $8N$.

Let p be the 4th-ranked point in some $\text{top4}(\vec{v})$. Such a configuration corresponds to a half-plane whose boundary contains p and which contains exactly four colors.

For each p , the number of such possible color sets is at most 7 (or often 4 if p 's color appears elsewhere). Thus the total number is $O(N)$.

Detailed reasoning: Consider rotating a half-plane around p . If other points of the same color exist and must be excluded, rotation is limited to $\leq 180^\circ$, and the number of candidate color sets is ≤ 4 . If full 360° rotation is allowed, the number is ≤ 7 .

8. Near-Linear Algorithm

We propose the following:

1. Compute convex hulls of each color and discard interior points.
2. Rotate $\vec{v}$ through 360° and enumerate candidates for A and C from $\text{top4}(\vec{v})$ and $\text{top4}(-\vec{v})$. The number of candidates is $O(N)$.
3. Sort all $\vec{AC}$ candidates by angle.
4. For each $\vec{AC}$, compute B and D from $\text{top4}(\vec{v})$ and $\text{top4}(-\vec{v})$ where $\vec{v} = \text{rot}(\vec{AC})$.

The total time is $f(N) + O(N \log N)$, where $f(N)$ is the time to enumerate all $\text{top4}(\vec{v})$.

9. Enumerating $\text{top4}(\vec{v})$

We describe an $O(N(\log N)^2)$ method.

In a segment tree over the color range:

Each node $[l, r)$ stores:

- **top4**: the $\text{top4}(\vec{v})$ for colors in $[l, r)$,
- **v_next**: the next direction where this node's **top4** or any child's **top4** changes.

The monoid operation:

- Merge children's **top4** sets (8 points) to produce the parent's 4-point **top4**.
- Compute **v_next** as the earliest breakpoint among child nodes or from changes in the merged hull.

During rotation, advance $\vec{v}$ to the root's **v_next**, then update all nodes whose **v_next** matches it.

Each node updates $O(n)$ times where n is its number of points. Thus the total is $O(N(\log N)^2)$.

Problem M. Many Approaches

We will refer to the squares and the park as **cells** and the **grid**, respectively. Also, we call a ***march*** a **scenario**.

Offline Solution

We first describe a solution that is allowed to read all queries in advance and answer them offline.

We extend both the people and the cells by M to the left and right. That is, assume the cells are indexed $-M, -M+1, \dots, N+M-1$, and initially person i stands on cell i . When answering how many people are on a given cell, we count only people $0, 1, \dots, N-1$.

First, we simulate the scenario over the entire array A . We then design a data structure that supports the following queries:

1. Return the cell index where person x currently stands.
2. Return the minimum index of any person standing on cell x .

3. Return the maximum index of any person standing on cell x .
4. Move every person not on cell x one step closer to x .

Because movements never change the relative order of people, the difference between the positions of person $i + 1$ and person i is always 0 or 1. Moreover, once this difference becomes 0, it stays 0 forever. Thus, we maintain these differences d_i using a segment tree with range sums. Queries 1, 2, and 3 can be implemented via binary search on the segment tree, and Query 4 reduces to updating at most two points in the segment tree using Queries 1–3.

During the full simulation over A , we issue Query 4 for $x = A_1, A_2, \dots, A_M$ in order.

We can answer interval-scenario queries by issuing appropriate data-structure queries at the correct times during this simulation. We denote the scenario for $(A_L, A_{L+1}, \dots, A_R)$ by $A[L, R]$.

Determining the destination of person P under scenario $A[L, R]$

Immediately before processing $x = A_L$, we query (using Query 2) for some person standing on cell P ; call this person y . Because we extended the number of people and cells, such a y always exists. We can then identify the movement of person y with that of person P under scenario $A[L, R]$. Thus, immediately after processing $x = A_R$, Query 1 gives the final cell of y , which is the answer.

Counting how many people end up on cell P after scenario $A[L, R]$

Immediately after processing $x = A_R$, Queries 2 and 3 give the minimum and maximum person indices on cell P , say l and r . Immediately before processing $x = A_L$, we use Query 1 to find the positions l' and r' of persons l and r , respectively. The set of people who end up on cell P under scenario $A[L, R]$ corresponds to the set of people who were initially within positions $[l', r']$ before A_L . In the actual scenario, only people 1 through N exist at the beginning, each occupying one position, so the answer is the size of the intersection of intervals $[l', r']$ and $[1, N]$.

Online Conversion

These queries can also be processed online. Interval-scenario queries do not modify the data structure, so it suffices to simulate the entire scenario for A in advance, and to access the data structure at specific times. Persistence is all that is required; the above method works with a persistent segment tree. This solution runs in $O((N + M + Q) \log(N + M))$ time and space.

Remark

The reason for requiring online queries is to distinguish this problem from a naive approach using a balanced binary tree, where one would insert “relevant” people into the structure per query without extending people or cells.